Introduction

In this notebook, we'll implement implement the forward pass of an SSM (State Space Model) using recursion and convolution based approaches. We'll also compare the two approaches in terms of speed and memory usage.

You will need GPU for this notebook which can be enabled via changing the runtime.

You can copy your solutions from the `q_coding_ssm_forward_cpu` notebook for the first part of the notebook.

Imports

```
import time
import math
import matplotlib.pyplot as plt

import torch
import numpy as np
import torch.nn.functional as F

torch.set_default_device('cuda')
```

SSM Update Rule

We consider an RNN described by the update:

$$h_{t+1} = W\, h_t + U\, x_t + b$$

for $t = 0, 1, \ldots, T-1$. The variables are:

- $h_t \in R^H$, the hidden state at time t.
- $x_t \in R^{N \times D}$, the input at time t.
- $W \in R^{H \times H}$, the recurrent weight matrix.
- $U \in R^{H \times D}$, the input projection matrix.
- $b \in R^H$, the bias vector.

N is the batch size, D is the input dimension, and H is the hidden state dimension. We assume $h_0 = 0$, the all-zero vector of dimension H.

Below you will implement the forward pass for the SSM using recursion based approach. The `unrolled_ssm_forward` function will take weights W, U, b and input x and return the hidden states h across different time steps.

```python
def unrolled_ssm_forward(W, U, b, x):
    """
    Unroll the linear RNN in time:
        h_{t+1} = W h_t + U x_t + b
    with initial h_0 = 0.

    Args:
      W: (H, H) weight matrix
      U: (H, D) input projection
      b: (H,)   bias
      x: (N, T, D) input sequence over T steps
    Returns:
      h_all: (N, T, H) hidden states for t=1..T
             (h_all[t] corresponds to h_{t+1} in the usual notation).
    """
    N, T, D = x.shape
    H = W.shape[0]

    # Initialize the output tensor
    h_all = torch.zeros((N, T, H), device=x.device)

    # Initialize the first hidden state
    h_prev = torch.zeros((N, H), device=x.device)  # h_0 = 0

    # Loop through each time step
    for t in range(T):
        # Get current input
        x_t = x[:, t, :]  # shape (N, D)

        # Compute next hidden state: h_{t+1} = W h_t + U x_t + b
        h_next = torch.mm(h_prev, W.T) + torch.mm(x_t, U.T) +
b.unsqueeze(0)

        # Store the result
        h_all[:, t, :] = h_next

        # Update the hidden state for next iteration
        h_prev = h_next

    return h_all
```

Convolution Based Implementation

In the previous problem, you showed that the forward pass of an SSM can be implmemented using a convolution operation. In this problem, you will implement the forward pass of an SSM using a convolution based approach. You can assume that T is a power of 2.

You will implement two functions

- `make_conv_kernel(W, T)`: This function will take the recurrent weight matrix W and the number of time steps T and return the convolution kernel K. Given that T is a power of 2, you can implement this using a divide and conquer based approach.
- `conv_ssm_forward(W, U, b, x)`: This function will take weights W, U, b and input x and return the hidden states h across different time steps.

```
def make_conv_kernel(W, T):
    """
    Build a 3D kernel tensor K of shape (H, H, T) we will use when
implementing
    the ssm forward pass using conv1d.

    Args:
      W: (H, H) weight matrix
      T: scalar

    Returns:
      kernel_for_conv: (H, H, T) tensor
    """
    H = W.shape[0]
    # Initialize the kernel with zeros
    K = torch.zeros((H, H, T), device=W.device)

    # Base case: K[:, :, 0] = I (identity matrix)
    K[:, :, 0] = torch.eye(H, device=W.device)

    # If T = 1, we're done
    if T == 1:
        return K

    # For T > 1, we'll use a divide and conquer approach
    # First handle the case where T is a power of 2
    if (T & (T - 1)) == 0:  # Check if T is a power of 2
        # Start with K[:, :, 1] = W
        K[:, :, 1] = W.clone()

        # Double the time steps until we reach T
        t = 2
        while t < T:
            # K[:, :, t:2t] = K[:, :, t-1] @ K[:, :, 1:t+1]
            # Using matrix multiplication for the convolution
            for i in range(t, min(2*t, T)):
                K[:, :, i] = torch.mm(K[:, :, t-1], K[:, :, i - (t-
1)])
            t *= 2
    else:
        # For non-power of 2, we'll use a more general but less
efficient approach
        K[:, :, 0] = torch.eye(H, device=W.device)
        W_power = W.clone()
```

```
        for t in range(1, T):
            K[:, :, t] = W_power
            W_power = torch.mm(W, W_power)

    return K

def conv_ssm_forward(W, U, b, x):
    """
    Convolution-based forward pass for an RNN with weight matrix W.

    RNN update:  h_{t+1} = W h_t + U x_t + b

    Args:
      W: (H, H) weight matrix
      U: (H, D) input projection
      b: (H,) bias
      x: (N, T, D) => N = batch, T = time steps, D = input dim

    Returns:
      h_all: (N, T, H) hidden states for t=1..T
    """
    N, T, D = x.shape
    H = W.shape[0]

    # Create the convolution kernel
    K = make_conv_kernel(W, T)  # (H, H, T)

    # Project the input: Ux + b
    # x shape: (N, T, D) -> (N, T, H) after projection
    Ux_plus_b = torch.einsum('hd,ntd->nth', U, x) + b[None, None, :]
# (N, T, H)

    # Reshape for batch matrix multiplication
    # We'll process each output dimension separately
    h_all = torch.zeros((N, T, H), device=x.device)

    # For each output dimension
    for i in range(H):
        # Get the kernel for this output dimension: (H, T)
        kernel = K[i]  # (H, T)

        # Reshape input for conv1d: (N, H, T)
        input_reshaped = Ux_plus_b.permute(0, 2, 1)  # (N, H, T)

        # Reshape kernel for conv1d: (H, 1, T)
        kernel_reshaped = kernel.unsqueeze(1)  # (H, 1, T)

        # Apply conv1d for this output dimension
        # We need to process each input channel separately and sum the
results
```

```
        conv_out = torch.zeros((N, 1, 2*T-1), device=x.device)

        for j in range(H):
            # Process one input channel at a time
            input_slice = input_reshaped[:, j:j+1, :]  # (N, 1, T)
            kernel_slice = kernel_reshaped[j:j+1]  # (1, 1, T)

            # Add to the output
            conv_out += F.conv1d(
                input_slice,
                kernel_slice,
                padding=T-1,
                groups=1
            )

        # Take only the valid part of the convolution (last T
elements)
        # and store in h_all
        h_all[:, :, i] = conv_out[:, 0, T-1:2*T-1]  # (N, T)

    return h_all
```

Sanity Check

We can compare the outputs of the two implementations to check if they are consistent.

```
def sanity_check():
    T = 8   # number of time steps
    H = 4   # hidden dimension
    D = 3   # input dimension
    N = 2

    torch.manual_seed(0)

    W = torch.randn(H, H) * 0.1
    U = torch.randn(H, D) * 0.1
    b = torch.randn(H) * 0.1

    x = torch.randn(N, T, D)

    h_unrolled = unrolled_ssm_forward(W, U, b, x)
    h_conv = conv_ssm_forward(W, U, b, x)

    diff = (h_unrolled - h_conv).abs().max()
    print("Unrolled h(t):")
    print(h_unrolled)
    print("\nConv-based h(t):")
    print(h_conv)
```

```
    print("\nMax absolute difference:", diff.item())

sanity_check()
```

Implementation Complexity

We can compare the two implementations in terms of efficiency. Particularly, we will compare the time taken by the two implementations to compute the hidden states for a given input and weights with varying number of time steps.

```
import ipywidgets as widgets
from ipywidgets import interact

def measure_runtime(method_fn, W, U, b, x, warmup=1, repeats=10):
    # Warm-up runs (ignored in timing):
    for _ in range(warmup):
        method_fn(W, U, b, x)

    # Timed runs:
    start = time.time()
    for _ in range(repeats):
        method_fn(W, U, b, x)
    end = time.time()

    avg_time = (end - start) / repeats
    return avg_time


def run():
    from tqdm import tqdm
    T_values = [8, 32, 128, 256, 512]
    H_values = [4, 8, 16, 32, 64, 128, 256, 512]
    T_values_cache = {}
    times_unrolled_vs_T_cache = {}
    times_conv_vs_T_cache = {}

    for H in tqdm(H_values, desc=f"Hidden Dim (H)", leave=False):
      # We'll keep D, N fixed
      D = 32
      N = 32

      T_values = [8, 32, 128, 256, 512]

      # Build random U, b
      U = torch.randn(H, D)*0.1
      b = torch.randn(H)*0.1

      times_unrolled_vs_T = []
```

```
      times_conv_vs_T = []

      for T in tqdm(T_values, desc=f"Sequence Length (T), H={H}",
leave=False):

          diag_vals = torch.randn(H)*0.05
          W = torch.randn(H, H)*0.05
          x = torch.randn(N, T, D)

          t_unrolled = measure_runtime(unrolled_ssm_forward, W, U, b,
x)

          t_conv = measure_runtime(conv_ssm_forward, W, U, b, x)

          times_unrolled_vs_T.append(t_unrolled)
          times_conv_vs_T.append(t_conv)

      T_values_cache[H] = T_values
      times_unrolled_vs_T_cache[H] = times_unrolled_vs_T
      times_conv_vs_T_cache[H] = times_conv_vs_T
    return T_values_cache, times_unrolled_vs_T_cache,
times_conv_vs_T_cache

T_values_cache, times_unrolled_vs_T_cache, times_conv_vs_T_cache =
run()

@interact(H=widgets.FloatLogSlider(min=2, max=9, base=2, value=4,
step=1))
def interactive_benchmark(H):
    """
    Compare unrolled vs. diagonal-convolution RNN forward for various
T,
    at a chosen hidden dimension H from the slider.
    """
    H = int(H)
    T_values = T_values_cache[H]
    T_unrolled = times_unrolled_vs_T_cache[H]
    T_conv = times_conv_vs_T_cache[H]

    # Plot
    plt.figure(figsize=(6,4))
    plt.plot(T_values, T_unrolled, label="Unrolled", marker='o')
    plt.plot(T_values, T_conv, label="Conv", marker='s')
    plt.title(f"Runtime vs T, H={H}")
    plt.xlabel("Time Steps (T)")
    plt.ylabel("Runtime (sec)")
    plt.yscale('log')
    plt.grid(True)
    plt.legend()
    plt.show()
```

Question 6

What do you observe about the runtime of the two implementations as T and H increase? Do you observe the same trend as the CPU notebook? Explain your reasoning.

Introducing structure: Diagonal Weight Matrics

We can optimize the convolution implemntation via adding a constraint on the W matrix ensuring it is diagonal. We can make the convolution implementation more efficient by leveraging depthwise convolutions for implementing the forward operation.

```
def make_diag_depthwise_kernel(W, T):
    """
    Construct a depthwise 1D conv kernel for W with shape (H,H). W is
a diagonal matrix.

    Args:
      W: (H, H) diagonal matrix
      T: (int) number of time steps

    Return:
      kernel of shape (H, 1, T), which can be used in depthwise
convolution
    """
    # Check if W is a diagonal matrix
    assert W.dim() == 2 and W.size(0) == W.size(1), "W must be a
square matrix"
    assert torch.allclose(W, torch.diag(torch.diag(W))), "W must be a
diagonal matrix"

    H = W.size(0)

    # Get the diagonal elements of W
    diag_elements = torch.diag(W)  # Shape: (H,)

    # Create a tensor to hold the powers of W's diagonal elements
    # We'll compute W^0, W^1, W^2, ..., W^{T-1} for each diagonal
element
    powers = torch.arange(T, device=W.device, dtype=torch.float32)  #
0 to T-1

    # Compute powers for each diagonal element
    # Shape: (H, T) where each row is [1, w_i, w_i^2, ..., w_i^{T-1}]
    kernel = diag_elements.view(-1, 1).pow(powers.view(1, -1))

    # Reshape to (H, 1, T) for depthwise convolution
    kernel = kernel.unsqueeze(1)  # Add dimension for input channels
(1)
```

```
    return kernel

def diag_conv_ssm_forward(W, U, b, x):
    """
    Convolution-based forward pass for an RNN with diagonal W

    RNN update:  h_{t+1} = W h_t + U x_t + b
    but W is diagonal, so h_{t+1}(i) = diagW[i]*h_t(i) + [U x_t + b]
(i).

    Args:
      W: (H, H) [diagonal in practice]
      U: (H, D)
      b: (H,)
      x: (N, T, D) => N = batch, T = time steps, D = input dim

    Returns:
      h_all: (N, T, H) hidden states for t=1..T
    """
    N, T, D = x.shape
    H = W.shape[0]

    # Create the depthwise convolution kernel
    kernel = make_diag_depthwise_kernel(W, T)  # (H, 1, T)

    # Project the input: Ux + b
    # x shape: (N, T, D) -> (N, T, H) after projection
    Ux_plus_b = torch.einsum('hd,ntd->nth', U, x) + b[None, None, :]
# (N, T, H)

    # Reshape for depthwise convolution: (N, T, H) -> (N, H, T, 1)
    Ux_plus_b_reshaped = Ux_plus_b.permute(0, 2, 1).unsqueeze(-1)  #
(N, H, T, 1)

    # Reshape kernel for depthwise conv2d: (H, 1, T) -> (H, 1, T, 1)
    kernel_reshaped = kernel.unsqueeze(-1)  # (H, 1, T, 1)

    # Apply depthwise convolution using conv2d
    # We use groups=H for depthwise convolution
    conv_out = F.conv2d(
        Ux_plus_b_reshaped,
        kernel_reshaped,
        groups=H,
        padding=(T-1, 0)
    )  # (N, H, 2T-1, 1)

    # Take only the valid part of the convolution (last T elements)
    # and reshape to (N, H, T)
    h_all = conv_out.squeeze(-1)[:, :, T-1:2*T-1]
```

```
    # Transpose to get (N, T, H)
    h_all = h_all.permute(0, 2, 1)

    return h_all
```

Optimizing the recurrent implementation

Similarly, we can also optimize the recurrent implementation by using the diagonal nature of the W matrix.

```
def diag_unrolled_ssm_forward(W, U, b, x):
    """
    Unrolled forward pass for an RNN with diagonal W

    RNN update:  h_{t+1} = W h_t + U x_t + b
    but W is diagonal, so h_{t+1}(i) = diagW[i]*h_t(i) + [U x_t + b]
(i).

    Args:
      W: (H, H) [diagonal in practice]
      U: (H, D)
      b: (H,)
      x: (N, T, D) => N = batch, T = time steps, D = input dim

    Returns:
      h_all: (N, T, H) hidden states for t=1..T
    """
    N, T, D = x.shape
    H = W.shape[0]

    # Initialize the output tensor
    h_all = torch.zeros((N, T, H), device=x.device)

    # Get the diagonal elements of W
    diag_W = torch.diag(W)  # (H,)

    # Initialize the hidden state
    h_prev = torch.zeros((N, H), device=x.device)  # h_0 = 0

    # Project the input: Ux + b for all time steps
    # x shape: (N, T, D) -> (N, T, H) after projection
    Ux_plus_b = torch.einsum('hd,ntd->nth', U, x) + b[None, None, :]
# (N, T, H)

    # Unroll the RNN
    for t in range(T):
```

```
        # Compute next hidden state using element-wise multiplication
for diagonal W
        h_next = diag_W * h_prev + Ux_plus_b[:, t, :]

        # Store the result
        h_all[:, t, :] = h_next

        # Update the hidden state for next iteration
        h_prev = h_next

    return h_all
```

Sanity Check

We can compare the outputs of the two implementations to check if they are consistent similar to above.

```
def diag_sanity_check():
    T = 8   # number of time steps
    H = 4   # hidden dimension
    D = 3   # input dimension
    N = 2

    torch.manual_seed(0)

    W = torch.eye(H) * 0.1 #### WE USE A DIAGONAL MATRIX HERE

    U = torch.randn(H, D) * 0.1
    b = torch.randn(H) * 0.1

    x = torch.randn(N, T, D)

    h_unrolled = diag_unrolled_ssm_forward(W, U, b, x)

    h_conv = diag_conv_ssm_forward(W, U, b, x)

    diff = (h_unrolled - h_conv).abs().max()
    print("Unrolled h(t):")
    print(h_unrolled)
    print("\nConv-based h(t):")
    print(h_conv)
    print("\nMax absolute difference:", diff.item())

diag_sanity_check()
```

Measure Runtime with Optimization

Similarly, we will measure performance optimization with the diagonalized implementations.

```
def diag_run():
    T_values_cache = {}
    times_unrolled_vs_T_cache = {}
    times_conv_vs_T_cache = {}

    for H in [4, 8, 16, 32, 64, 128, 256, 512]:
      # We'll keep D, N fixed
      D = 32
      N = 512

      T_values = [8, 32, 128, 256, 512]

      # Build random U, b
      U = torch.randn(H, D)*0.1
      b = torch.randn(H)*0.1

      times_unrolled_vs_T = []
      times_conv_vs_T = []

      for T in T_values:
          diag_vals = torch.randn(H)*0.05

          W = torch.eye(H, H)*0.05 #### WE USE A DIAGONAL MATRIX HERE

          x = torch.randn(N, T, D)

          t_unrolled = measure_runtime(diag_unrolled_ssm_forward, W,
U, b, x)

          t_conv = measure_runtime(diag_conv_ssm_forward, W, U, b, x)

          times_unrolled_vs_T.append(t_unrolled)
          times_conv_vs_T.append(t_conv)

      T_values_cache[H] = T_values
      times_unrolled_vs_T_cache[H] = times_unrolled_vs_T
      times_conv_vs_T_cache[H] = times_conv_vs_T
    return T_values_cache, times_unrolled_vs_T_cache,
times_conv_vs_T_cache

diag_T_values_cache, diag_times_unrolled_vs_T_cache,
diag_times_conv_vs_T_cache = diag_run()

@interact(H=widgets.FloatLogSlider(min=2, max=9, base=2, value=4,
step=1))
def interactive_benchmark(H):
```

```
    """
    Compare unrolled vs. diagonal-convolution RNN forward for various
T,
    at a chosen hidden dimension H from the slider.
    """
    H = int(H)
    T_values = diag_T_values_cache[H]
    T_unrolled = diag_times_unrolled_vs_T_cache[H]
    T_conv = diag_times_conv_vs_T_cache[H]

    # Plot
    plt.figure(figsize=(6,4))
    plt.plot(T_values, T_unrolled, label="Unrolled", marker='o')
    plt.plot(T_values, T_conv, label="Diag-Conv", marker='s')
    plt.title(f"Runtime vs T, H={H} diagonal weights")
    plt.xlabel("Time Steps (T)")
    plt.ylabel("Runtime (sec)")
    plt.grid(True)
    plt.legend()
    plt.show()
```

Question 7

What do you observe here? How do your findings different from the unstructured matrix case? Explain your reasoning.